Data S1

Python and MATLAB code for Appendices S1–S5

Eco-evolutionary predator–prey model with temperature-dependent trait matching

Authors	[Author names]
Affiliation	[Institution, address]
Contact	[e-mail address]
Archive	`DataS1.zip`
Licence	[to be chosen, e.g. MIT for code]

Contents

1 Overview **1**

2 File list **1**

2.1 Top level 2

2.2 Python (`python/`) 2

2.3 MATLAB (`matlab/`) 3

3 Requirements and running **3**

4 Conventions **4**

5 Reproducibility **4**

6 Known limitations **4**

1. Overview

The archive contains two independent implementations, in Python and in MATLAB, of every calculation behind Appendices S1–S5:

- closed-form equilibria and Jacobians, and their symbolic and numerical checks;
- Routh–Hurwitz coefficients and subsystem determinants;
- Hopf points, phase-lag coefficients and the exact phase relation;
- long simulations of the full nonlinear model with a convergence test;
- the one-species submodels, random parameter searches and the temperature × enrichment scenarios.

Each script writes its tables (CSV), logs (TXT) and figures (PDF/PNG) to `outputs/python/` or `outputs/matlab/`. A further script recomputes the published-model Jacobians in Cortez (2018), Appendices S4 and S5, from their stated equations.

The two implementations share the parameter draws of the random searches (`data/`), so every reported number, including the random-search counts, is the same in both languages.

2. File list

Top level

File	Description
`README.md`	Quick start.
`Metadata_S1.pdf`	This document.
`data/random_sets_prey_only.csv`	6000 parameter draws (seed 7) for the prey-only search, Appendix S4 Section S1.3. Columns `T,K,h,e,m,bx,by,sr,sc,sa,so,amax,rmax`.
`data/random_sets_h0.csv`	1500 draws with $h = 0$ for the no-handling-time search, Appendix S4 Section S2.1.

Python (`python/`)

File	Description
`ecoevo/model.py`	Baseline parameters, right-hand side, closed-form equilibrium and derived quantities, reduced and full Jacobians, finite-difference Jacobian.
`ecoevo/stability.py`	Routh–Hurwitz coefficients, Δ_2, Δ_3, subsystem determinants, grouped unstable subsystems, stability maps, Hopf roots, the V_x window quadratic, prey-only stability patterns.
`ecoevo/phaselag.py`	Eigenvector lags, the exact lag formula, surviving C_1/C_2 terms and C_3.
`ecoevo/simulate.py`	Integration (LSODA), period and Fourier-phase lags, two-window convergence statistics.
`_common.py`	Output directory, logging and the `--quick` flag.
`appS1_jacobian_checks.py`	Appendix S1: symbolic Jacobian (sympy) vs closed forms; equilibrium Jacobian vs finite differences on 200 random parameter sets; determinant, $\tilde{J}_{33}^{\text{eff}}$ and fast-limit identities.
`appS2_routh_hurwitz.py`	Appendix S2: subsystem determinants, closed-form a_i, $a_1a_2 - a_3$ window, Figure S1.
`appS3_phase_lags.py`	Appendix S3: Hopf points, exact collapse of C_1 and C_2, Table S3, simulations for Table S4, Figure S1.
`appS4_applications.py`	Appendix S4: predator-only and prey-only submodels, Table S1, random searches (writes `data/`), $\lvert\Delta T\rvert$ threshold, scenario grid (Table S2, Figure S1).
`appS5_figure_parameters.py`	Appendix S5: equilibria, Jacobians and unstable subsystems of every parameter set; scenario table; attractor ranges (needs `appS3_sims.json`).
`cortez2018_crosscheck.py`	Recomputes the Jacobians printed in Cortez (2018), Appendices S4–S5.
`run_all.py`	Runs all scripts in order; accepts `--quick`.
`requirements.txt`	Python dependencies.

MATLAB (matlab/)

File	Description
ecoevo_params.m, ecoevo_rates.m, ecoevo_rhs.m	Parameters (name–value overrides), unscaled rates, ODE right-hand side.
ecoevo_equilibrium.m	Closed-form equilibrium and derived quantities.
ecoevo_jacobian_reduced.m, ecoevo_jacobian_general.m, ecoevo_jacobian_fd.m	Equilibrium Jacobian (V factored out), Jacobian at any state, finite-difference Jacobian.
scale_jac.m, hurwitz.m, leading_eig.m	Scaling by V_x, V_y; a_1–a_4, Δ_2, Δ_3; leading eigenvalue.
subsystem_dets.m, unstable_groups.m	Principal minors; grouped unstable subsystems.
stability_map.m, hopf_roots.m, lead_switches.m, window_quadratic.m, prey_only_pattern.m	Stability maps, Hopf points (fzero), stability switches, V_x window, prey-only patterns.
eig_lags.m, lag_terms.m, fundamental_lags.m	Eigenvector lags, $C_1/C_2/C_3$ terms, simulated Fourier-phase lags.
simulate_model.m, attractor_stats.m	Integration (default ode45) and two-window statistics.
newton_solve.m	Damped Newton solver (replaces fsolve, so no Optimization Toolbox is needed).
logline.m, outdir.m, save_fig.m	Logging, output directory, figure export.
appS1_jacobian_checks.m … cortez2018_crosscheck.m	One script per appendix, as for Python.
run_all.m	Runs all scripts; honours QUICK and NOFIG.

3. Requirements and running

Python. Python ≥3.9 with numpy, scipy, sympy and matplotlib. The code was tested with Python 3, numpy 2.4.4, scipy 1.17.1, sympy 1.14.0 and matplotlib 3.10.8.

```
cd python
pip install -r requirements.txt
python run_all.py            # full run
python run_all.py --quick    # fast test (coarse grids, unconverged simulations)
```

MATLAB. R2016b or later. Base MATLAB only: no Optimization, Signal Processing or Symbolic Math Toolbox is required. If the Symbolic Math Toolbox is present, appS1_jacobian_checks.m adds a symbolic check. All computations were tested with GNU Octave 8.4.

```
cd matlab
run_all                      % full run
QUICK = true; run_all        % fast test
NOFIG = true; run_all        % no figures (headless Octave)
```

Run python/appS4_applications.py once before the MATLAB version of Appendix S4, unless data/ is already present; it is included in the archive.

Runtimes. Python on one core:

- Appendix S3 simulations: about 30–60 minutes.
- Appendix S4, including the 7500 random parameter sets: about 2 minutes.
- All other scripts: under 1 minute each.

In Octave 8.4, one quick-mode set-A simulation (8000 time units) took about 2.5 minutes, so the full Appendix S3 simulations take several hours there; MATLAB is typically faster.

4. Conventions

State variables. $X = (N, P, x, y)$ is indexed 1–4 (MATLAB) or 0–3 (Python): prey density, predator density, mean prey trait, mean predator trait.

Parameters. Code names map to symbols as follows (baseline values in brackets): `rmax` $r_{\max}$ (1.2), `K` K (7; varied), `amax` $a_{\max}$ (0.8), `h` h (0.3), `e` e (0.6), `m` m (0.2), `bx` β_x (0.7), `by` β_y (0.5), `sr` s_r (1.5), `sc` s_c (2), `sa` s_a (1.2), `so` s_o (1), `T` T (25), `Tr` T^*_{r0} (23), `Ta` T^*_{a0} (22).

Jacobians. "Reduced" Jacobians have V_x and V_y factored out of rows 3 and 4 (Appendix S1). Routh–Hurwitz coefficients are those of $\det(\lambda I - J)$.

Equilibria. The coexistence equilibrium is computed from its closed form (main analysis, Section 2.3), never by selecting a numerical root by index. Root-finding is used only for the submodels and the cross-check.

Lags. Lags are fractions of a period by which P, x or y lag N. The predicted lag is $-\arg(w_j/w_1)/2\pi$ of the leading eigenvector. The simulated lag is the phase difference of the fundamental Fourier component over an integer number of periods.

Unstable subsystems. Reported only when the whole Routh–Hurwitz group they belong to has the wrong sign (Appendix S5, Section S1).

5. Reproducibility

- **Random numbers.** Python uses NumPy generators `default_rng(1)` (Appendix S1) and `default_rng(7)` (Appendix S4). The Appendix S4 draws are stored in `data/` and read by MATLAB.
- **Integration.** LSODA (Python) and `ode45` (MATLAB), with relative/absolute tolerances $10^{-9}/10^{-12}$ (sets A, B) and $10^{-10}/10^{-13}$ (set C).
- **Initial conditions.** $p\,(1 + \delta(1, -1, \frac{1}{2}, \frac{1}{2}))$ with $\delta = 10^{-2}$ (sets A, B) or 10^{-3} (set C).
- **Run lengths.** 60 000–80 000 time units (set A), 250 000 (set B) and 100 000 (set C).
- **Convergence.** Period and amplitude must agree to a relative 10^{-4} between the windows $[0.8, 0.9]$ and $[0.9, 1]$ of the run.
- **Quick mode.** Shortens runs and coarsens grids. Its simulated lags are not converged and are for testing only.

Expected results. A correct installation reproduces the values in Table 1.

6. Known limitations

- The MATLAB figure code was written for MATLAB. Its computations were tested in headless GNU Octave 8.4, but its plotting could not be run in that environment (`NOFIG = true`). Octave's PDF export also needs Ghostscript, so `save_fig.m` warns rather than stops if export fails.
- MATLAB plots use $\log_{10}$-scaled axes via `imagesc`; Python uses logarithmic axes. The figures are equivalent but not pixel-identical.
- The set C attractor is a large-amplitude cycle that coexists with the stable equilibrium near the Hopf boundary. Its predator density falls to about 3.5×10^{-4} of the equilibrium value, so it should be read as a deterministic limit.
- Hopf points and stability switches are located by bracketing on finite grids. Features narrower than the grid spacing would be missed.

Script	Expected output
appS1	General Jacobian vs symbolic $\sim 10^{-16}$ (Python) or vs finite differences $\sim 10^{-10}$ (MATLAB); equilibrium Jacobian vs finite differences $\sim 10^{-9}$; $\lvert J\rvert$ at $K = 7$, $(V_x, V_y) = (0.37, 2.9)$: 1.174412×10^{-3}
appS2	$K = 7$ window $4.493 < V_x < 9.294$ at $V_y = 0$; unstable fractions 0.664, 0.020, 0 ($K = 50, 7, 5$; 241-point grid)
appS3	Set A Hopf $(V_x, V_y) = (4.4823, 0.01)$, $\omega = 0.4369$, $L_P = 0.2578$; $\sum C_1/C_3 = -0.0640$, $\sum C_2/C_3 = -3.0000$; set B Hopf $V_y = 0.060578$ at $V_x = 0.3$; set C attractor $T = 61.44$, $L_P = 0.1151$
appS4	Prey-only patterns U 2638, S 1028, US 647, SU 25, SUS 7; $h = 0$: 1035 tested, 0 unstable; $\lvert\Delta T\rvert$ threshold 2.183; switches 59.528, 4.493/9.294, 44.46
appS5	Set B equilibrium $(2.6088, 0.3197, -0.5963, 1.8804)$; 11 feasible scenarios
crosscheck	Saloniemi fig 3a entries 0.032 (printed 0.32); fig 3b analysed equilibrium has $\det J < 0$; Abrams & Matsuda $J_{33} = 0.085$; Fig 4C $J_{32} = +0.087$, $J_{33} = -0.057$

Table 1: Values reproduced by a correct installation.

Reference

Cortez, M. H. 2018. Genetic variation determines which feedbacks drive and alter predator–prey eco-evolutionary cycles. Ecological Monographs 88:353–371.